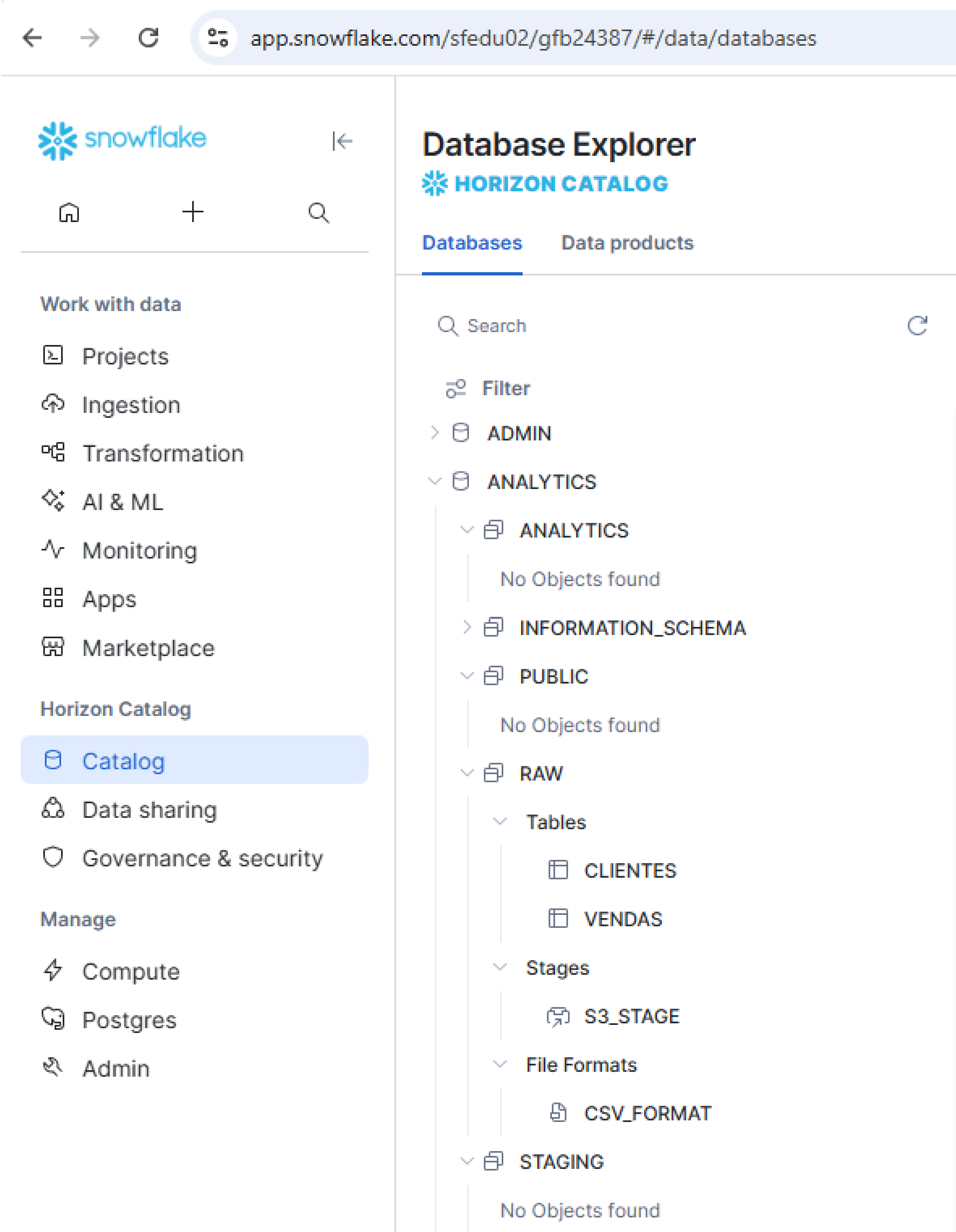
SNOWFLAKE

https://app.snowflake.com/

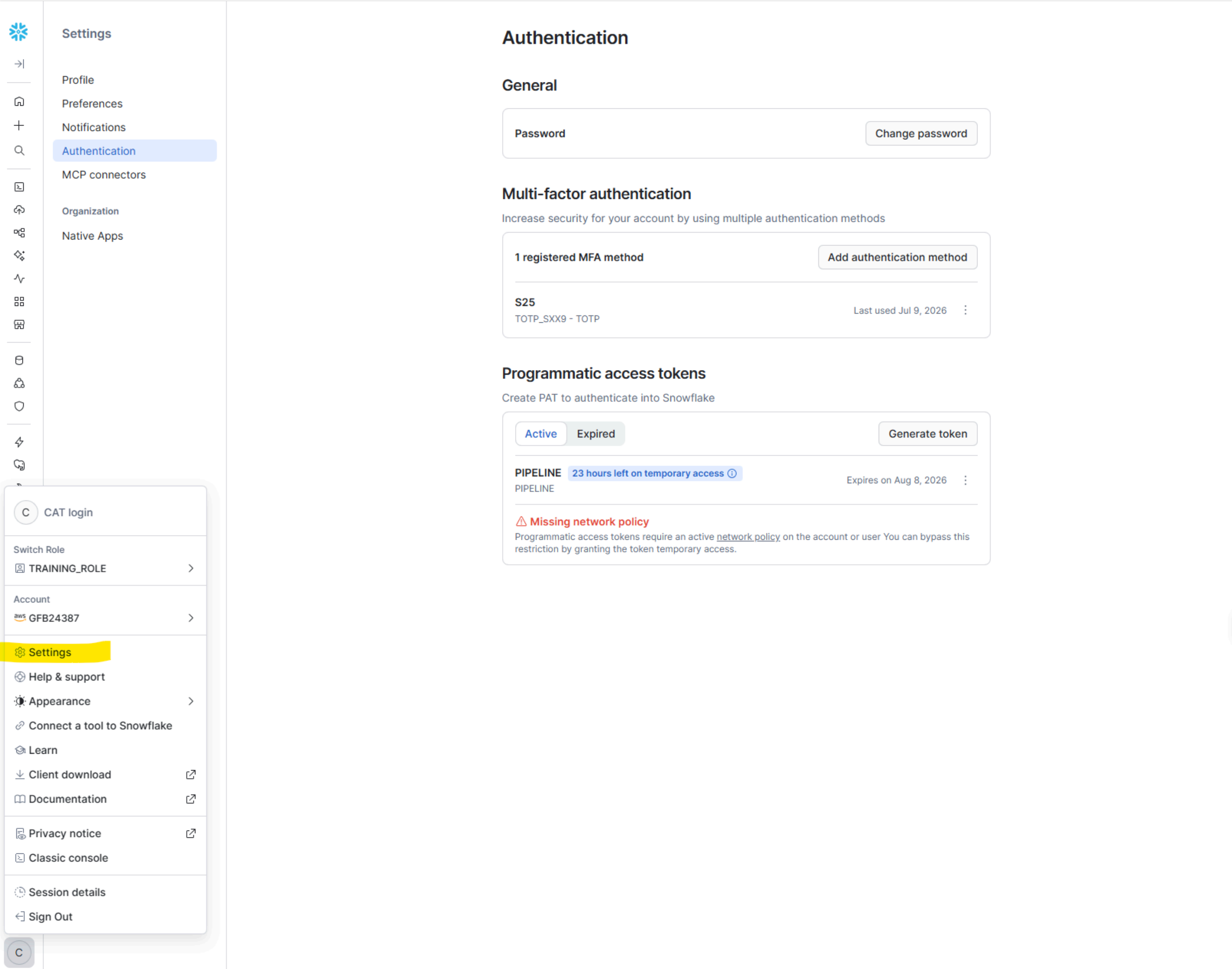


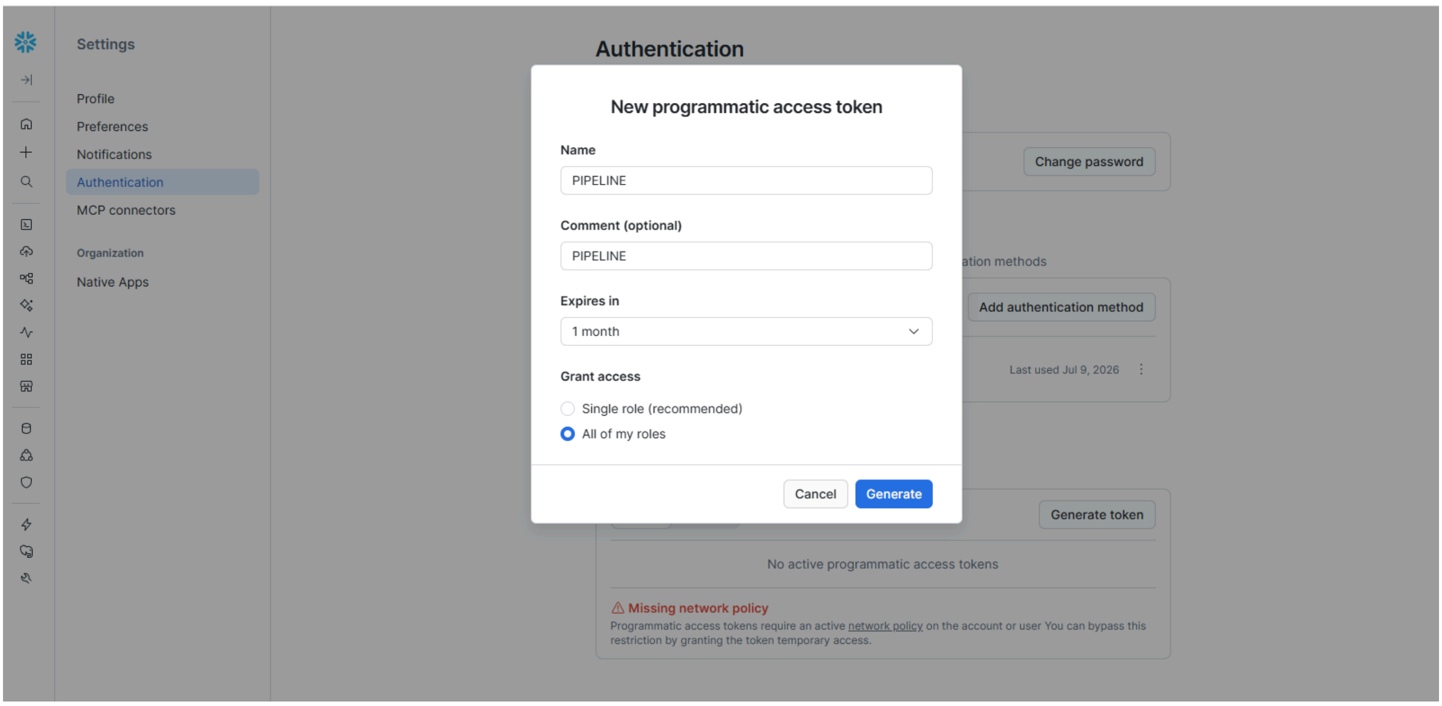
Criamos algumas tabelas e esquemas que serão utilizados:

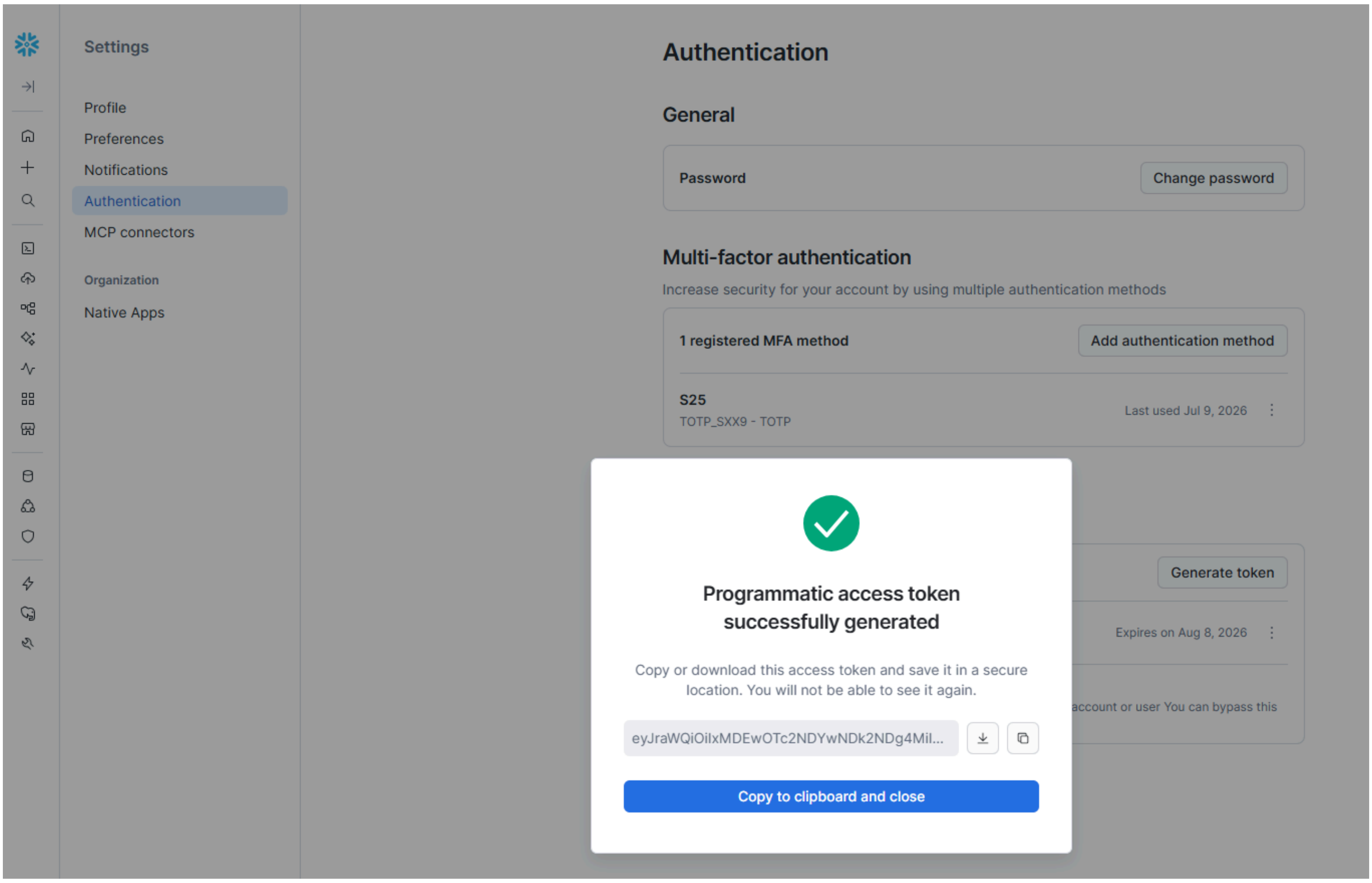
```sql
-- snowflake_setup.sql
-- Rode este script UMA VEZ no Snowflake (na interface web, aba Worksheets)
-- para preparar o ambiente antes de subir o Airflow.

-- 1) Warehouse (a "máquina" que processa) e database
CREATE WAREHOUSE IF NOT EXISTS COMPUTE_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60          -- desliga sozinho após 60s parado (economiza créditos)
  AUTO_RESUME = TRUE;

CREATE DATABASE IF NOT EXISTS ANALYTICS;

-- 2) Schemas: RAW recebe os dados do S3; ANALYTICS/STAGING são preenchidos pelo dbt
CREATE SCHEMA IF NOT EXISTS ANALYTICS.RAW;
CREATE SCHEMA IF NOT EXISTS ANALYTICS.STAGING;
CREATE SCHEMA IF NOT EXISTS ANALYTICS.ANALYTICS;

-- 3) Tabelas RAW (o Airflow vai carregar os CSVs do S3 aqui via COPY INTO)
CREATE TABLE IF NOT EXISTS ANALYTICS.RAW.CLIENTES (
    id_cliente STRING,
    nome       STRING,
    email      STRING,
    cidade     STRING
);

CREATE TABLE IF NOT EXISTS ANALYTICS.RAW.VENDAS (
    id_venda   STRING,
    id_cliente STRING,
    valor      STRING,
    data_venda STRING
);

-- 4) Formato de arquivo e External Stage apontando para o bucket S3
CREATE FILE FORMAT IF NOT EXISTS ANALYTICS.RAW.CSV_FORMAT
  TYPE = CSV
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  SKIP_HEADER = 1;            -- pula a linha de cabeçalho do CSV

-- Troque o URL, KEY e SECRET pelos seus valores reais.
CREATE STAGE IF NOT EXISTS ANALYTICS.RAW.S3_STAGE
  URL = 's3://meu-bucket-modelagem-ifg/'
  CREDENTIALS = (AWS_KEY_ID = 'SUA_KEY' AWS_SECRET_KEY = 'SEU_SECRET')
  FILE_FORMAT = ANALYTICS.RAW.CSV_FORMAT;
```

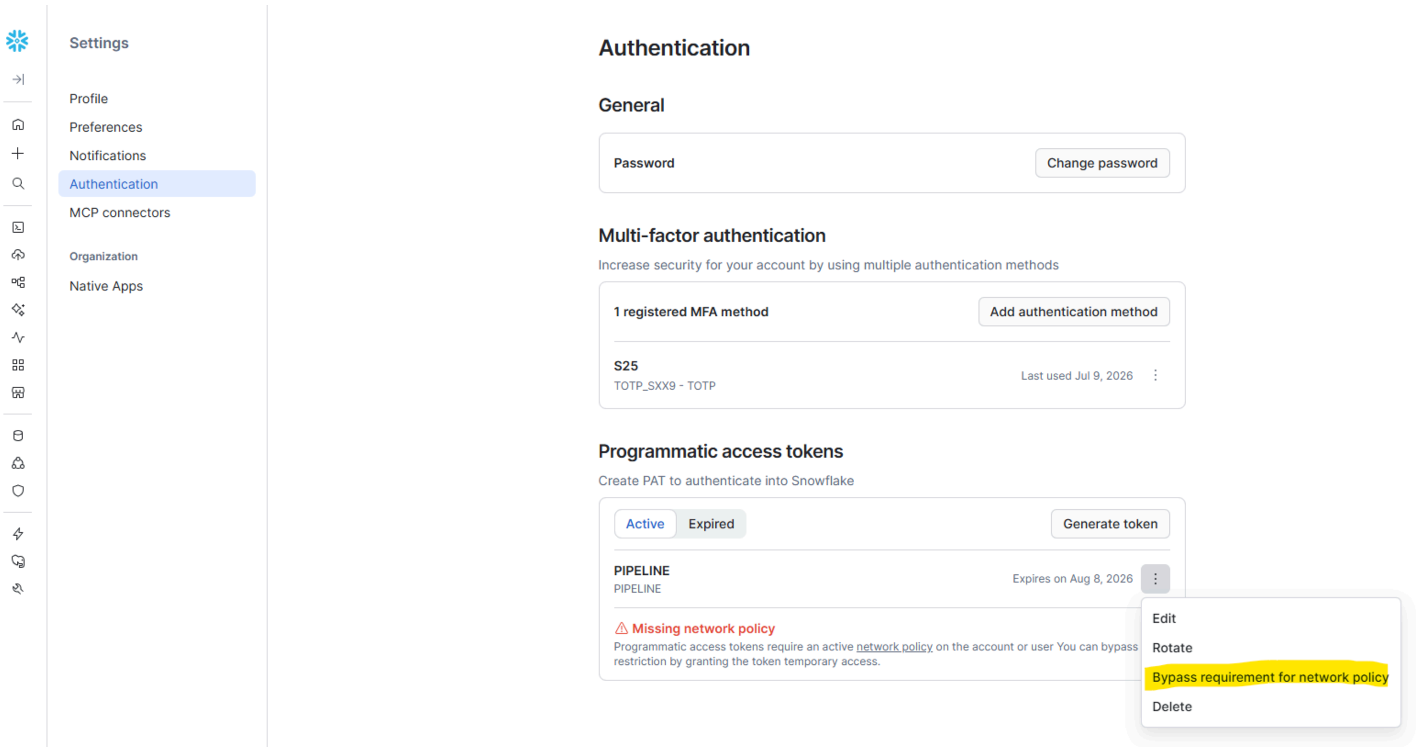
Permissão no Snowflake:

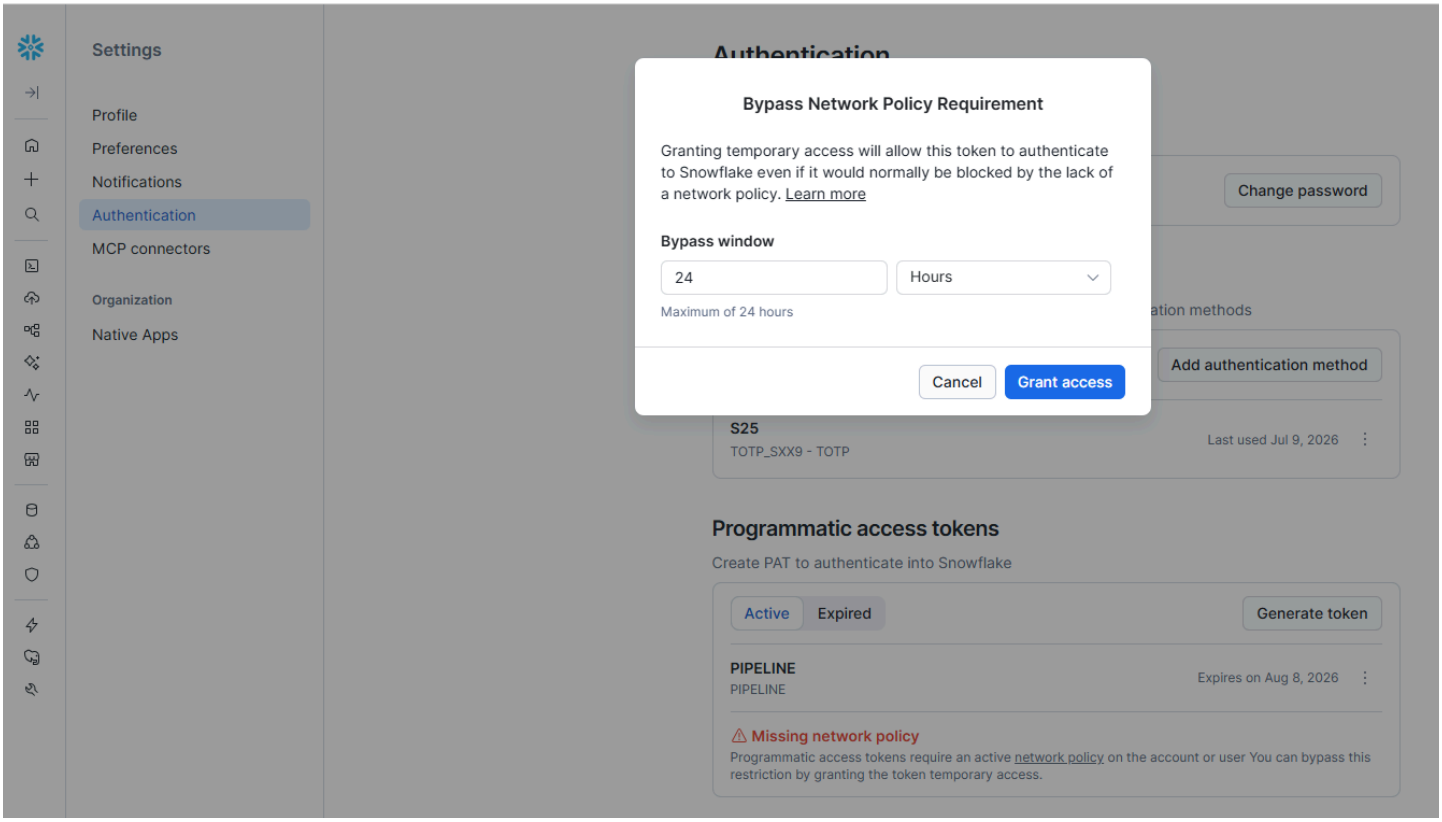
https://app.snowflake.com/sfedu02/gfb24387/settings/authentication






Settings
Profile
Preferences
Notifications
Authentication
MCP connectors
Organization
Native Apps
Authentication
General
Password
Change password
Multi-factor authentication
Increase security for your account by using multiple authentication methods
1 registered MFA method
Add authentication method
S25
TOTP_SXX9 - TOTP
Last used Jul 9, 2026
Generate token
Expires on Aug 8, 2026
account or user You can bypass this
Programmatic access token successfully generated
Copy or download this access token and save it in a secure location. You will not be able to see it again.
eyJraWQiOiIxMDEwOTc2NDYwNDk2NDg4Mil...
Copy to clipboard and close


Settings
Profile
Preferences
Notifications
Authentication
MCP connectors
Organization
Native Apps
Authentication
General
Password
Change password
Multi-factor authentication
Increase security for your account by using multiple authentication methods
1 registered MFA method
Add authentication method
S25
TOTP_SXX9 - TOTP
Last used Jul 9, 2026
Programmatic access tokens
Create PAT to authenticate into Snowflake
Active
Expired
Generate token
PIPELINE
PIPELINE
Expires on Aug 8, 2026
Missing network policy
Programmatic access tokens require an active network policy on the account or user You can bypass restriction by granting the token temporary access.
Edit
Rotate
Bypass requirement for network policy
Delete



Parâmetros que utilizamos no docker-compose:

```
# O ACCOUNT costuma ter o formato <org>-<conta>, ex: ab12345-xy67890
SNOWFLAKE_ACCOUNT=SFEDU02-GFB24387
SNOWFLAKE_USER=CAT
SNOWFLAKE_PASSWORD=XXXXXXXXXX
SNOWFLAKE_ROLE=TRAINING_ROLE
SNOWFLAKE_WAREHOUSE=CAT_WH
SNOWFLAKE_DATABASE=ANALYTICS
```

Parâmetros que utilizamos do DBT:

```
# dbt_project.yml — configura o projeto dbt
name: 'ifg_modelagem'
version: '1.0.0'
profile: 'ifg_modelagem'      # precisa bater com o nome no profiles.yml

model-paths: ["models"]

models:
  ifg_modelagem:
    staging:
      +materialized: view      # staging vira VIEW (leve, sempre lê o dado atual)
      +schema: staging
    marts:
      +materialized: table     # marts viram TABLE (materializada, rápida de consultar)
      +schema: analytics
```

```
# profiles.yml — diz ao dbt COMO conectar no Snowflake.
# Usa env_var() para ler as credenciais do ambiente (definidas no .env / docker-compose).
ifg_modelagem:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user: "{{ env_var('SNOWFLAKE_USER') }}"
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
      role: "{{ env_var('SNOWFLAKE_ROLE') }}"
      warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"
      database: "{{ env_var('SNOWFLAKE_DATABASE') }}"
      schema: analytics        # schema padrão de saída
      threads: 4               # roda até 4 models em paralelo
```

Copiar o Token gerado e colar no .env

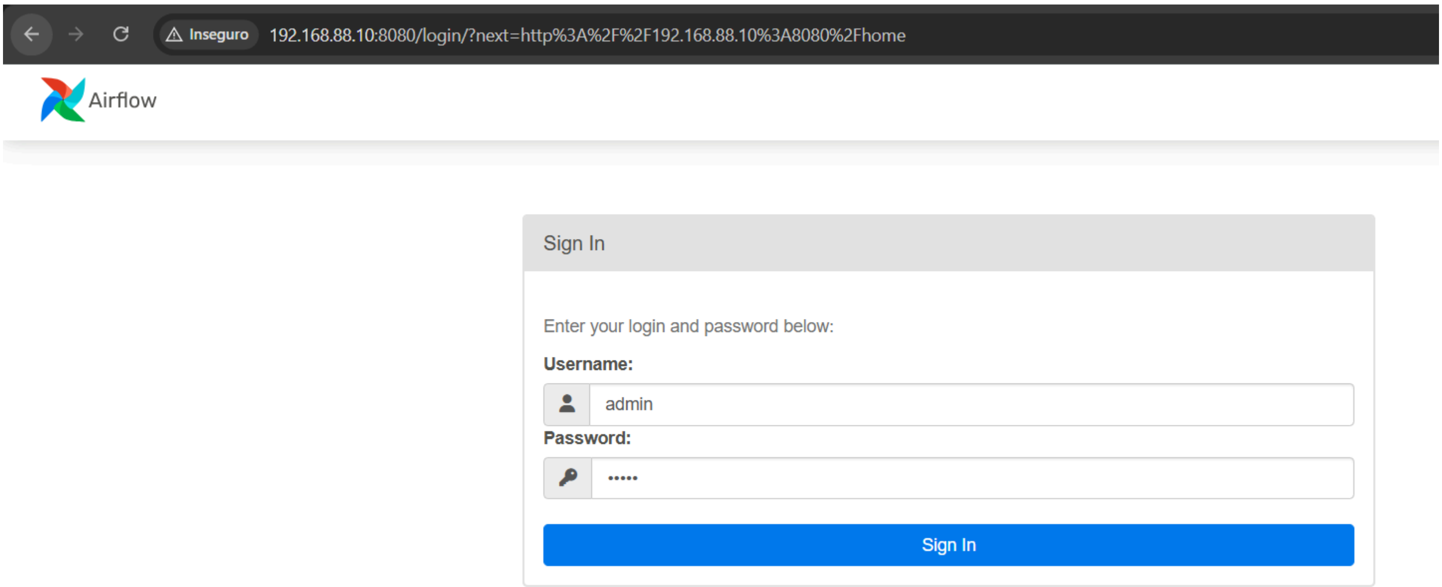
Será necessário reiniciar o docker compose.

AIRFLOW

Duplicar o arquivo .env.example para .env (esse não vai para o github).

Preencher as variáveis relacionadas ao S3, DBT e Snowflake.

http://192.168.88.10:8080/login/



Teste de conexão do Airflow com o S3

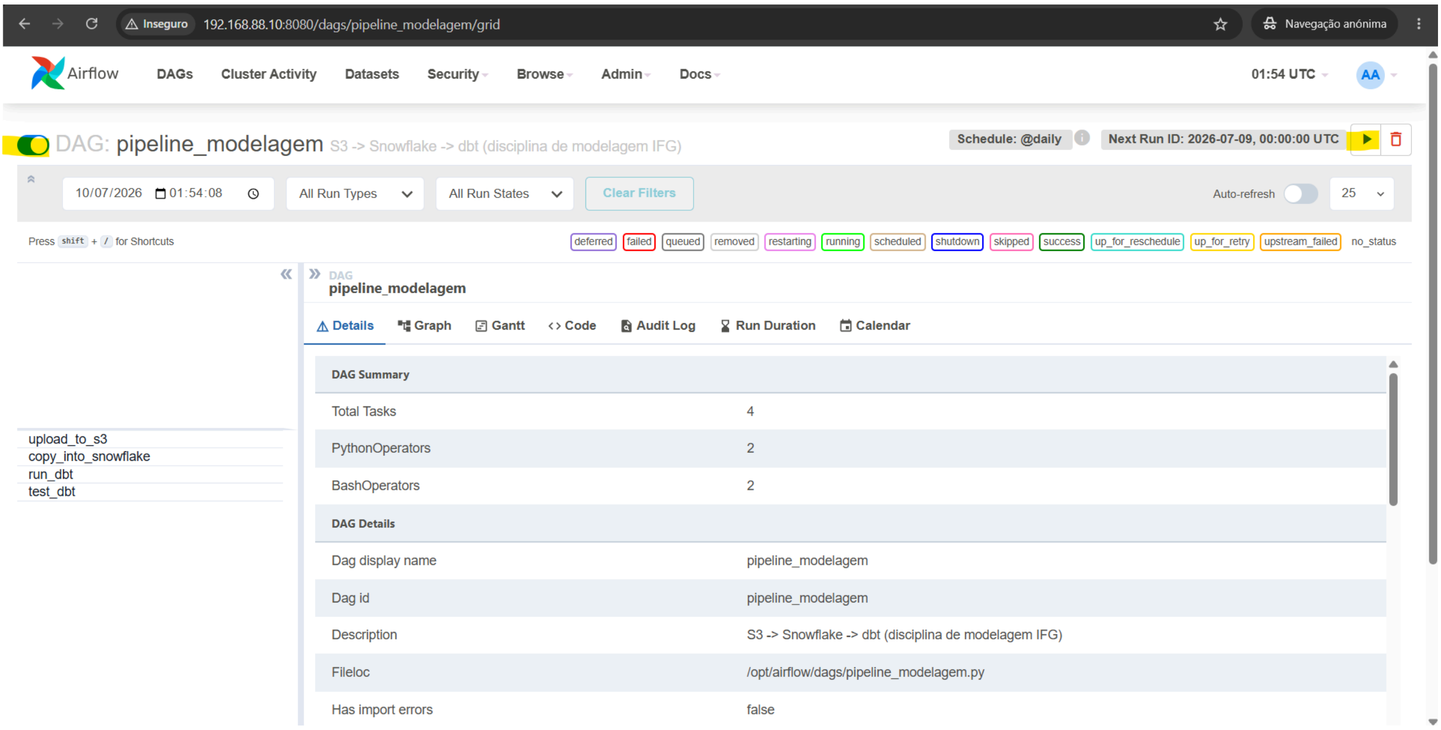
```
docker compose exec airflow-scheduler python -c "
import os, boto3
```

```
print('TOKEN presente:', bool(os.environ.get('AWS_SESSION_TOKEN')))
print(boto3.client('sts').get_caller_identity())
"
```

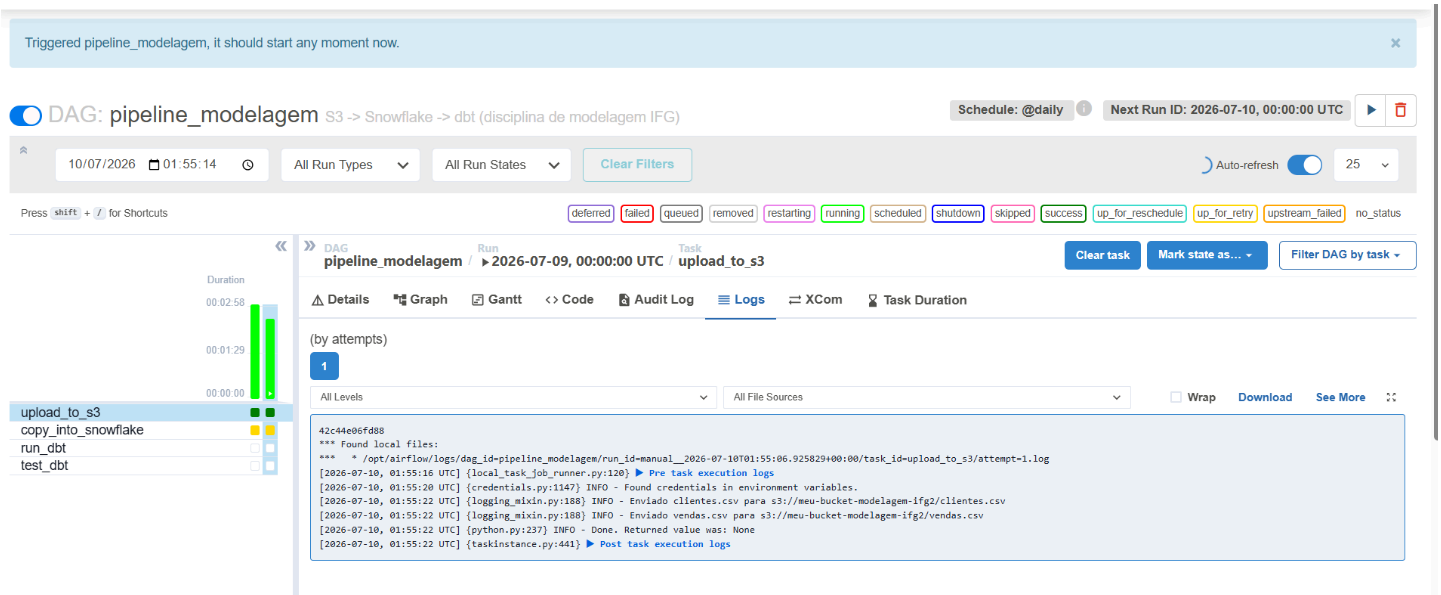
LOG de Sucesso

```
TOKEN presente: True
{'UserId': 'AROAZ5S7DVLKFD3VS3ZJD:user5101470=claudionoar@gmail.com', 'Account': '682025528020',
'Arn': 'arn:aws:sts::682025528020:assumed-role/voclabs/user5101470=claudionoar@gmail.com',
'ResponseMetadata': {'RequestId': 'cdbc8439-2f0c-4fae-b7ba-c89cbc9e3c55', 'HTTPStatusCode': 200,
'HTTPHeaders': {'x-amzn-requestid': 'cdbc8439-2f0c-4fae-b7ba-c89cbc9e3c55',
'x-amz-sts-extended-request-id': 'MTp1cy1lYXN0LTE6UzoxNzgzNTU3MDAxMjMyOkc6WGtWWlpvYlM=',
'content-type': 'text/xml', 'content-length': '480', 'date': 'Thu, 09 Jul 2026 00:30:01 GMT'}, 'RetryAttempts': 0}}
root@lab-ifg:/vagrant/tutorial-modelagem-v3# docker compose exec airflow-scheduler python -c "
import os, boto3
print('TOKEN presente:', bool(os.environ.get('AWS_SESSION_TOKEN')))
print(boto3.client('sts').get_caller_identity())
```

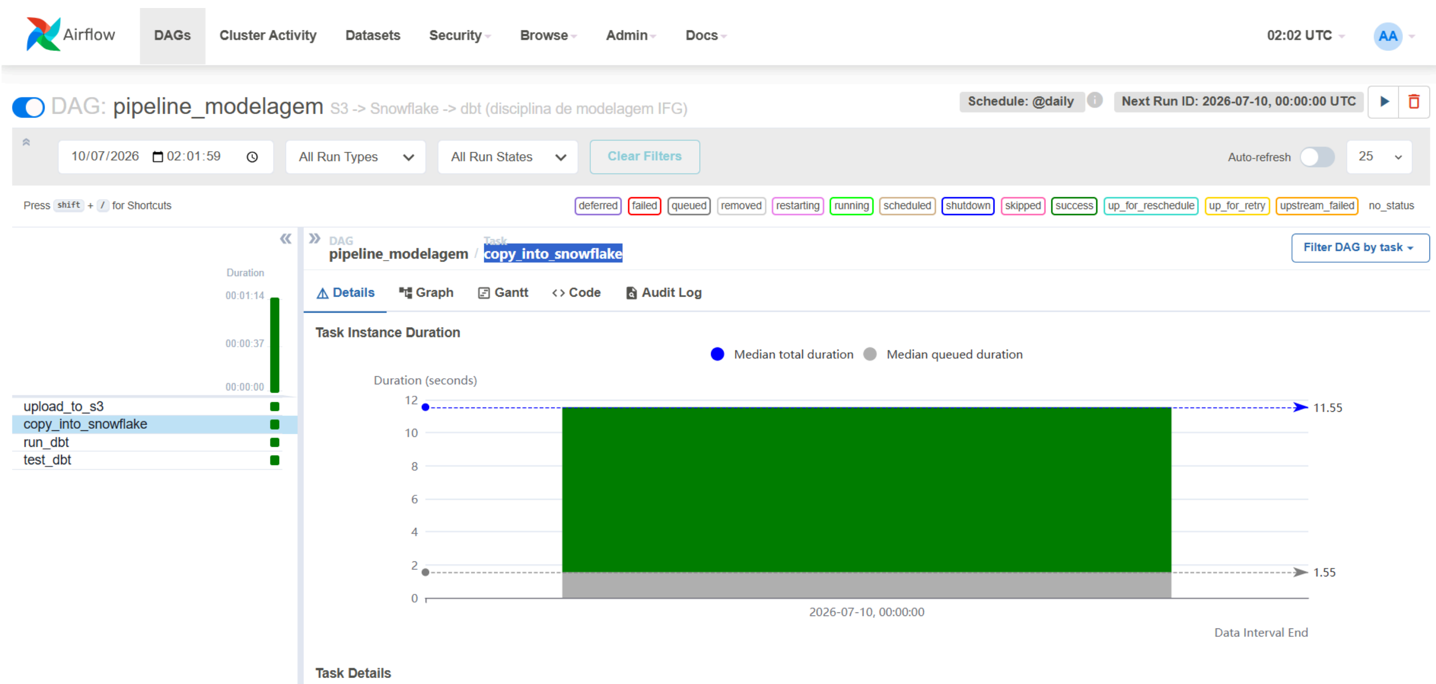
http://192.168.88.10:8080/dags/pipeline_modelagem/grid



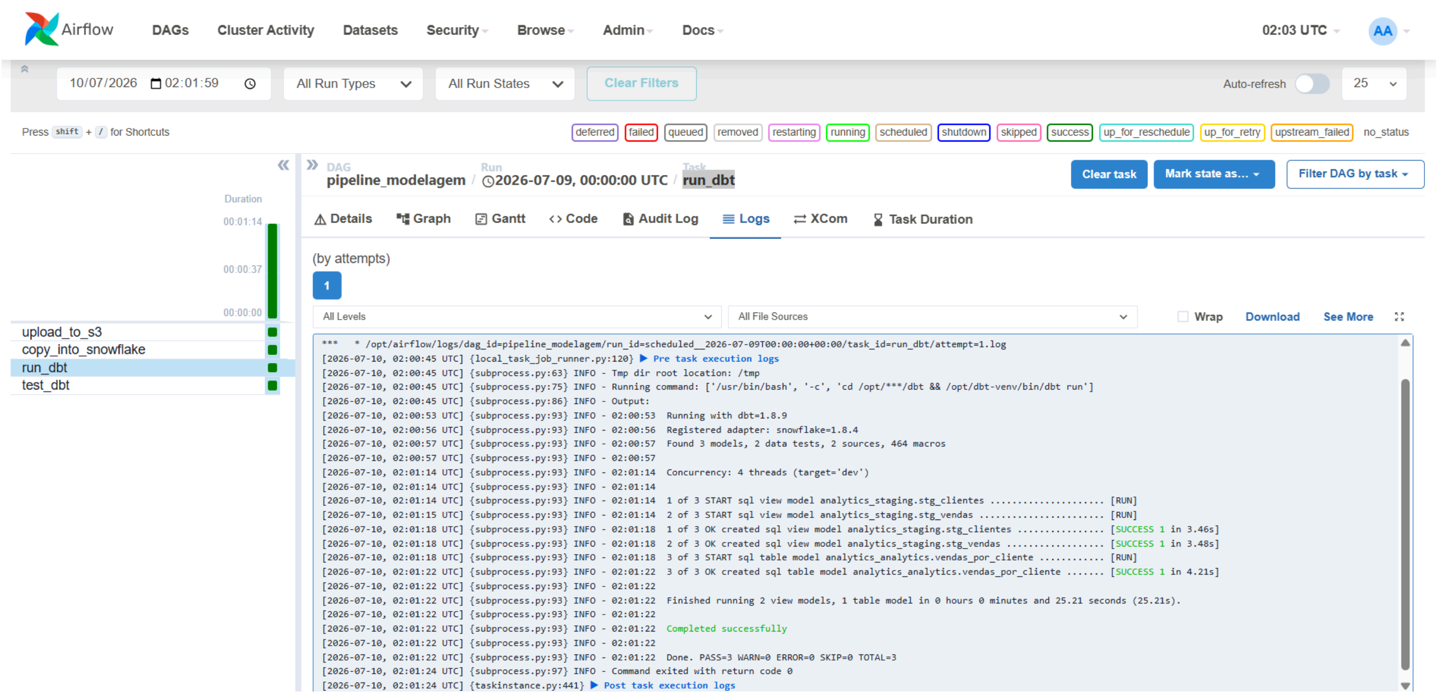
TASK upload_s3



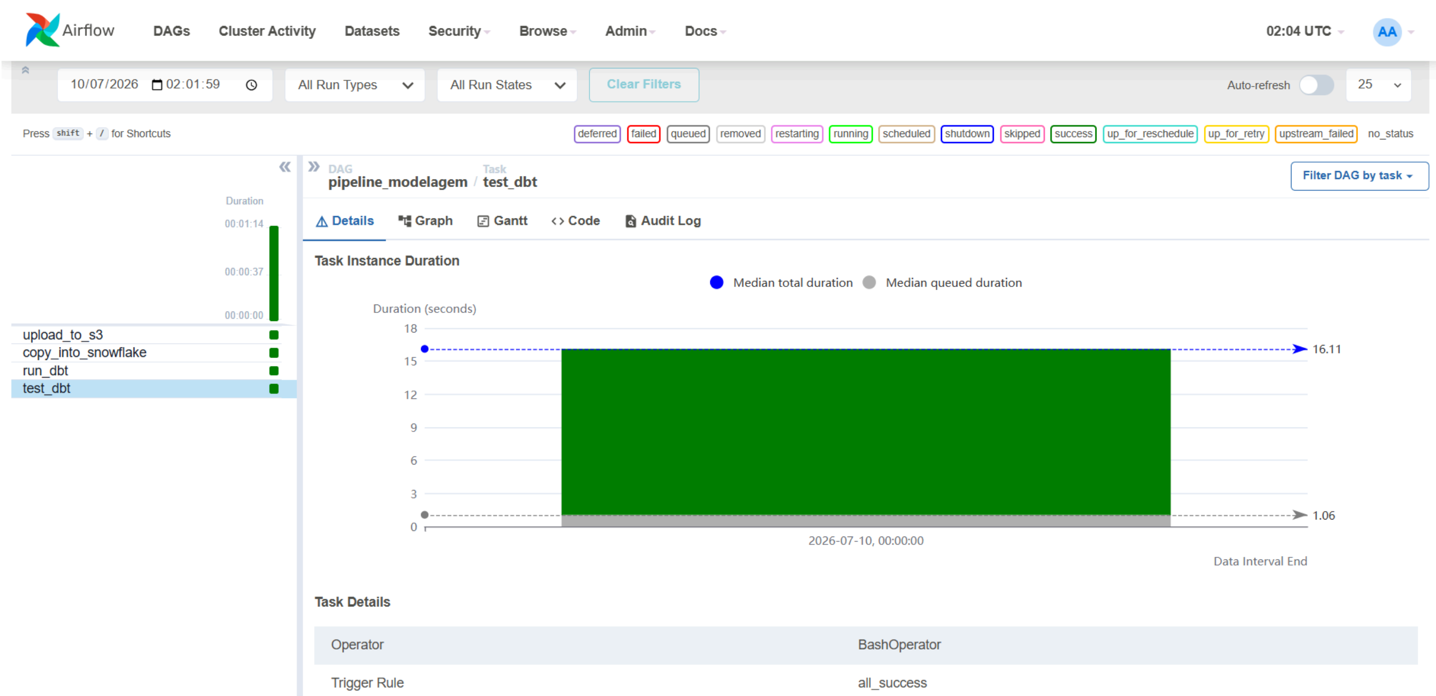
TASK copy_into_snowflake



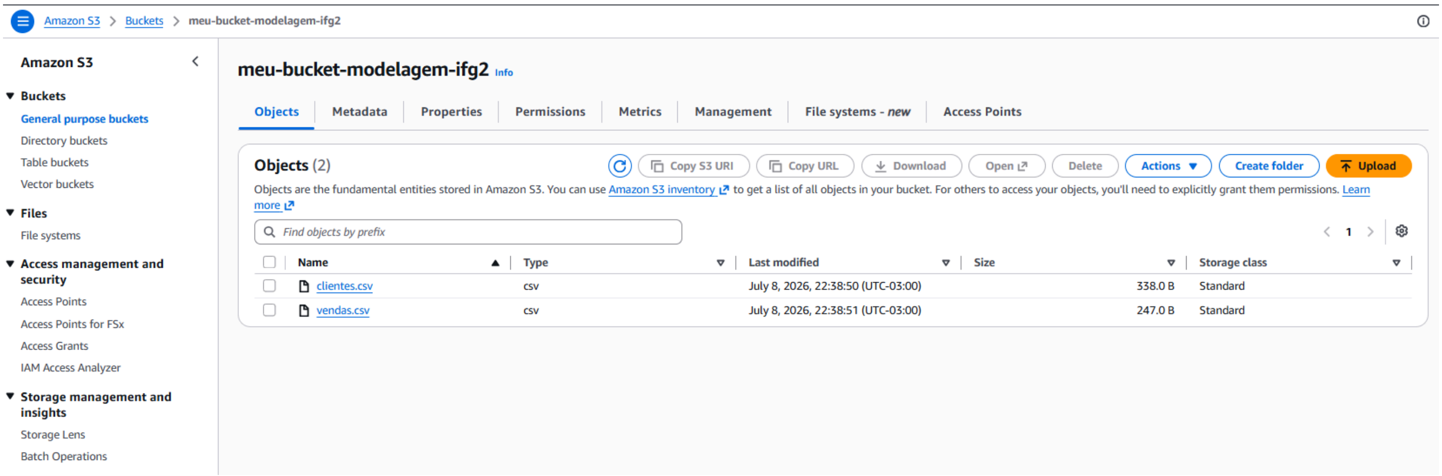
TASK run_dbt

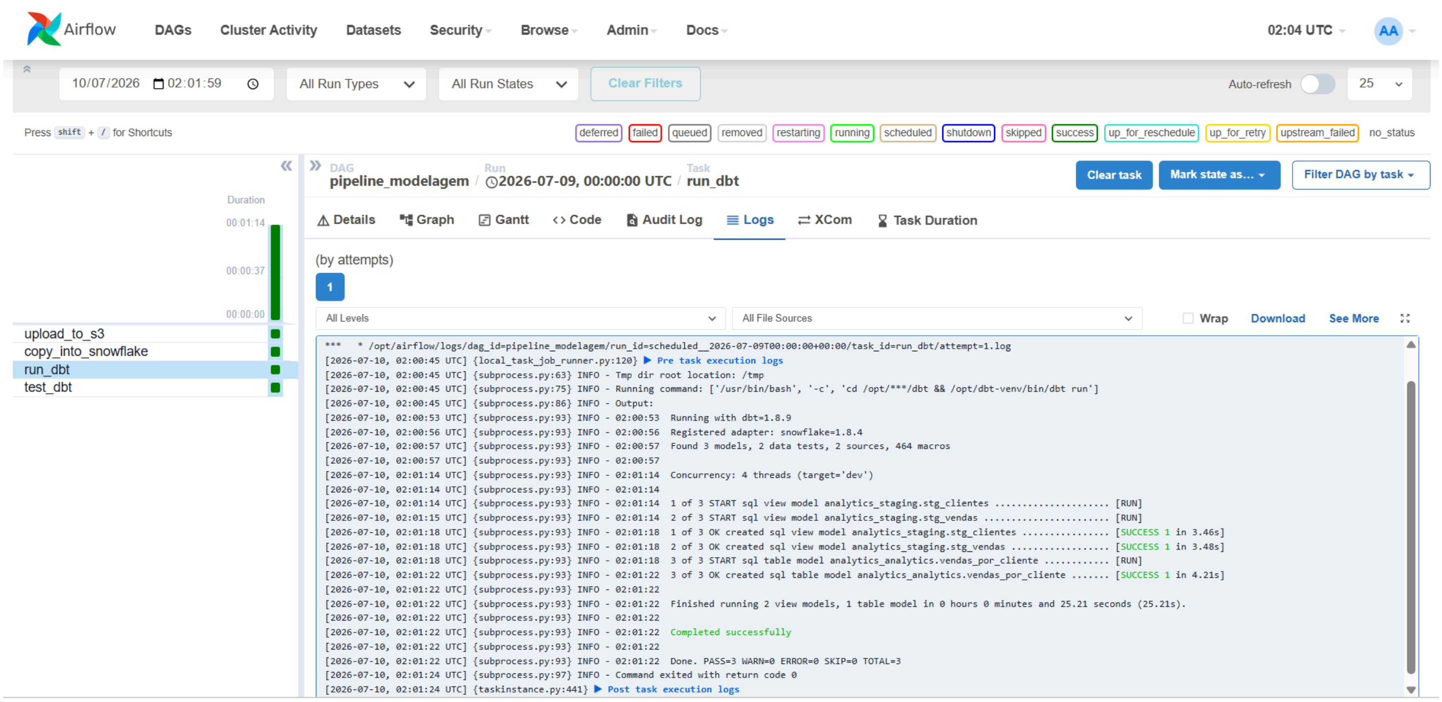


TASK test_dbt



Arquivos enviados via Airflow





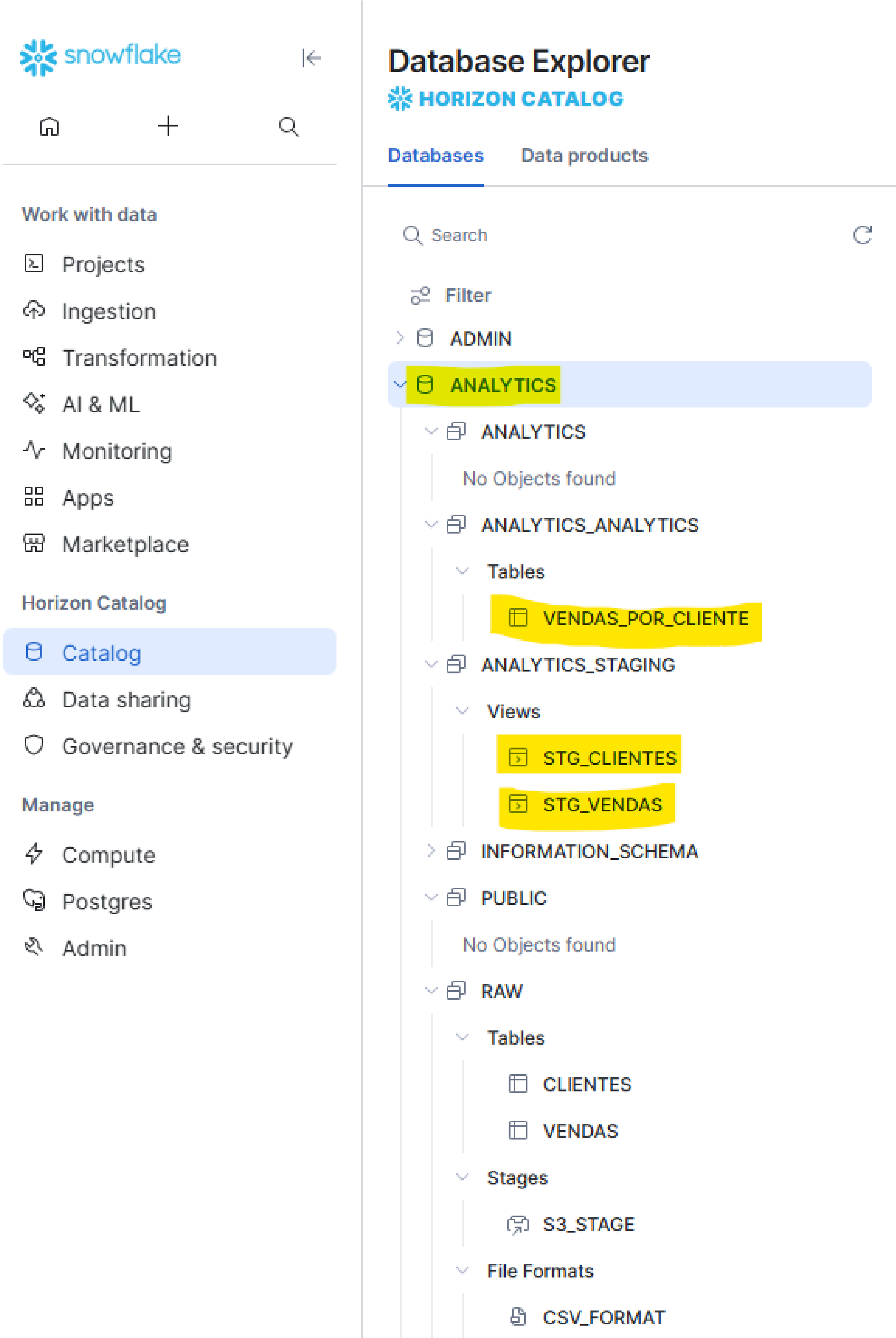
DBT

```
42c44e06fd88
*** Found local files:
***   *
/opt/airflow/logs/dag_id=pipeline_modelagem/run_id=scheduled__2026-07-09T00:00:00+00:00/task_id=run_dbt/attempt=1.log
[2026-07-10, 02:00:45 UTC] {local_task_job_runner.py:120} ▼ Pre task execution logs
[2026-07-10, 02:00:45 UTC] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=non-requeueable deps
ti=<TaskInstance: pipeline_modelagem.run_dbt scheduled__2026-07-09T00:00:00+00:00 [queued]>
[2026-07-10, 02:00:45 UTC] {taskinstance.py:2076} INFO - Dependencies all met for dep_context=requeueable deps
ti=<TaskInstance: pipeline_modelagem.run_dbt scheduled__2026-07-09T00:00:00+00:00 [queued]>
[2026-07-10, 02:00:45 UTC] {taskinstance.py:2306} INFO - Starting attempt 1 of 3
[2026-07-10, 02:00:45 UTC] {taskinstance.py:2330} INFO - Executing <Task(BashOperator): run_dbt> on 2026-07-09
00:00:00+00:00
[2026-07-10, 02:00:45 UTC] {standard_task_runner.py:64} INFO - Started process 289 to run task
[2026-07-10, 02:00:45 UTC] {standard_task_runner.py:90} INFO - Running: ['***', 'tasks', 'run', 'pipeline_modelagem',
'run_dbt', 'scheduled__2026-07-09T00:00:00+00:00', '--job-id', '13', '--raw', '--subdir',
'DAGS_FOLDER/pipeline_modelagem.py', '--cfg-path', '/tmp/tmpuf86t4lu']
[2026-07-10, 02:00:45 UTC] {standard_task_runner.py:91} INFO - Job 13: Subtask run_dbt
[2026-07-10, 02:00:45 UTC] {task_command.py:426} INFO - Running <TaskInstance: pipeline_modelagem.run_dbt
scheduled__2026-07-09T00:00:00+00:00 [running]> on host 42c44e06fd88
[2026-07-10, 02:00:45 UTC] {taskinstance.py:2648} INFO - Exporting env vars: AIRFLOW_CTX_DAG_OWNER='aluno_ifg'
AIRFLOW_CTX_DAG_ID='pipeline_modelagem' AIRFLOW_CTX_TASK_ID='run_dbt'
AIRFLOW_CTX_EXECUTION_DATE='2026-07-09T00:00:00+00:00' AIRFLOW_CTX_TRY_NUMBER='1'
AIRFLOW_CTX_DAG_RUN_ID='scheduled__2026-07-09T00:00:00+00:00'
```

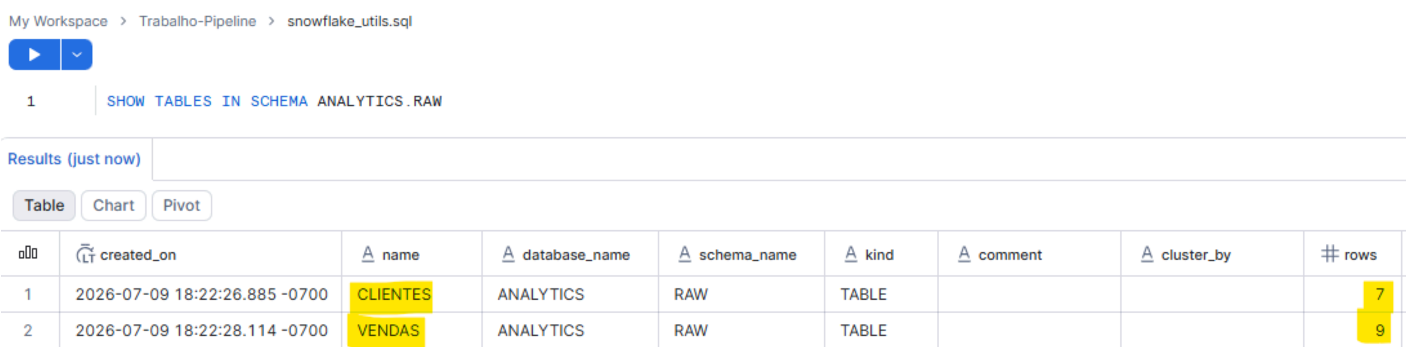
```
[2026-07-10, 02:00:45 UTC] {taskinstance.py:430} ▲▲▲ Log group end
[2026-07-10, 02:00:45 UTC] {subprocess.py:63} INFO - Tmp dir root location: /tmp
[2026-07-10, 02:00:45 UTC] {subprocess.py:75} INFO - Running command: ['/usr/bin/bash', '-c', 'cd /opt/***/dbt &&
/opt/dbt-venv/bin/dbt run']
[2026-07-10, 02:00:45 UTC] {subprocess.py:86} INFO - Output:
[2026-07-10, 02:00:53 UTC] {subprocess.py:93} INFO - 02:00:53  Running with dbt=1.8.9
[2026-07-10, 02:00:56 UTC] {subprocess.py:93} INFO - 02:00:56  Registered adapter: snowflake=1.8.4
[2026-07-10, 02:00:57 UTC] {subprocess.py:93} INFO - 02:00:57  Found 3 models, 2 data tests, 2 sources, 464 macros
[2026-07-10, 02:00:57 UTC] {subprocess.py:93} INFO - 02:00:57
[2026-07-10, 02:01:14 UTC] {subprocess.py:93} INFO - 02:01:14  Concurrency: 4 threads (target='dev')
[2026-07-10, 02:01:14 UTC] {subprocess.py:93} INFO - 02:01:14
[2026-07-10, 02:01:14 UTC] {subprocess.py:93} INFO - 02:01:14  1 of 3 START sql view model
analytics_staging.stg_clientes ..................... [RUN]
[2026-07-10, 02:01:15 UTC] {subprocess.py:93} INFO - 02:01:14  2 of 3 START sql view model analytics_staging.stg_vendas
....................... [RUN]
[2026-07-10, 02:01:18 UTC] {subprocess.py:93} INFO - 02:01:18  1 of 3 OK created sql view model
analytics_staging.stg_clientes ................ [SUCCESS 1 in 3.46s]
[2026-07-10, 02:01:18 UTC] {subprocess.py:93} INFO - 02:01:18  2 of 3 OK created sql view model
analytics_staging.stg_vendas .................. [SUCCESS 1 in 3.48s]
[2026-07-10, 02:01:18 UTC] {subprocess.py:93} INFO - 02:01:18  3 of 3 START sql table model
analytics_analytics.vendas_por_cliente ........... [RUN]
[2026-07-10, 02:01:22 UTC] {subprocess.py:93} INFO - 02:01:22  3 of 3 OK created sql table model
analytics_analytics.vendas_por_cliente ....... [SUCCESS 1 in 4.21s]
[2026-07-10, 02:01:22 UTC] {subprocess.py:93} INFO - 02:01:22
[2026-07-10, 02:01:22 UTC] {subprocess.py:93} INFO - 02:01:22  Finished running 2 view models, 1 table model in 0 hours
0 minutes and 25.21 seconds (25.21s).
[2026-07-10, 02:01:22 UTC] {subprocess.py:93} INFO - 02:01:22
[2026-07-10, 02:01:22 UTC] {subprocess.py:93} INFO - 02:01:22  Completed successfully
[2026-07-10, 02:01:22 UTC] {subprocess.py:93} INFO - 02:01:22
[2026-07-10, 02:01:22 UTC] {subprocess.py:93} INFO - 02:01:22  Done. PASS=3 WARN=0 ERROR=0 SKIP=0 TOTAL=3
[2026-07-10, 02:01:24 UTC] {subprocess.py:97} INFO - Command exited with return code 0
[2026-07-10, 02:01:24 UTC] {taskinstance.py:441} ▼ Post task execution logs
[2026-07-10, 02:01:24 UTC] {taskinstance.py:1206} INFO - Marking task as SUCCESS. dag_id=pipeline_modelagem,
task_id=run_dbt, run_id=scheduled__2026-07-09T00:00:00+00:00, execution_date=20260709T000000,
start_date=20260710T020045, end_date=20260710T020124
[2026-07-10, 02:01:24 UTC] {local_task_job_runner.py:243} INFO - Task exited with return code 0
[2026-07-10, 02:01:24 UTC] {taskinstance.py:3503} INFO - 1 downstream tasks scheduled from follow-on schedule check
[2026-07-10, 02:01:24 UTC] {local_task_job_runner.py:222} ▲▲▲ Log group end
```

SNOWFLAKE RESULTADO

https://app.snowflake.com/



SHOW TABLES IN SCHEMA ANALYTICS.RAW



	created_on	name	database_name	schema_name	kind	comment	cluster_by	rows
1	2026-07-09 18:22:26.885 -0700	CLIENTES	ANALYTICS	RAW	TABLE			7
2	2026-07-09 18:22:28.114 -0700	VENDAS	ANALYTICS	RAW	TABLE			9

LIST @ANALYTICS.RAW.S3_STAGE;

My Workspace > Trabalho-Pipeline > snowflake_utils.sql

```
LIST @ANALYTICS.RAW.S3_STAGE;
```

Results (just now)

Table Chart Pivot

	name	size	md5	last_modified
1	s3://meu-bucket-modelagem-ifg2/clientes.csv	336	5f22495ba8f9ec5e8cf1df04f7a48430	Fri, 10 Jul 2026 02:00:38 GMT
2	s3://meu-bucket-modelagem-ifg2/vendas.csv	246	bfbe2186efe35434bc32fb0109b33f34	Fri, 10 Jul 2026 02:00:39 GMT

SELECT COUNT(*) FROM ANALYTICS.RAW.CLIENTES; -- esperado: 7

```
SELECT COUNT(*) FROM ANALYTICS.RAW.CLIENTES
```

Results (just now)

Table Chart Pivot 1 row 233ms

	COUNT(*)
1	7

SELECT * FROM ANALYTICS.RAW.CLIENTES;

```
SELECT * FROM ANALYTICS.RAW.CLIENTES;
```

Results (just now)

Table Chart Pivot 7 rows 882ms

	ID_CLIENTE	NOME	EMAIL	CIDADE
1	1	Ana Silva	ana@email.com	Goiânia
2	2	Bruno Costa	bruno@email.com	Anápolis
3	3	Carla Dias	carla@email.com	Goiânia
4	4	Diego Reis	diego@email.com	Aparecida de Goiânia
5	5	Claduo	claudio@email.com	Aparecida de Goiânia
6	6	Jose Reis	jose@email.com	Aparecida de Goiânia
7	7	Maria	maria@email.com	Aparecida de Goiânia

SELECT COUNT(*) FROM ANALYTICS.RAW.VENDAS; -- esperado: 9

```
SELECT COUNT(*) FROM ANALYTICS.RAW.VENDAS
```

Results (just now)

Table Chart Pivot 1 row 433ms

	COUNT(*)
1	9

SELECT * FROM ANALYTICS.RAW.VENDAS;

```sql
SELECT * FROM ANALYTICS.RAW.VENDAS;
```

Results (just now)

Table | Chart | Pivot — 9 rows — 294ms

	ID_VENDA	ID_CLIENTE	VALOR	DATA_VENDA
1	101	1	150.00	2025-01-10
2	102	1	89.90	2025-01-15
3	103	2	320.50	2025-01-20
4	104	3	45.00	2025-02-01
5	105	3	210.75	2025-02-05
6	106	4	99.99	2025-02-10
7	107	5	99.99	2025-02-10
8	108	6	99.99	2025-02-10
9	109	7	99.99	2025-02-10

SELECT * FROM ANALYTICS.ANALYTICS_ANALYTICS.VENDAS_POR_CLIENTE;

```sql
SELECT * FROM ANALYTICS.ANALYTICS_ANALYTICS.VENDAS_POR_CLIENTE;
```

Results (just now)

Table | Chart | Pivot — 7 rows — 265ms

	ID_CLIENTE	NOME	CIDADE	QTD_COMPRAS	TOTAL_GASTO
1	1	Ana Silva	Goiânia	2	239.90
2	5	Claduo	Aparecida de Goiânia	1	99.99
3	6	Jose Reis	Aparecida de Goiânia	1	99.99
4	3	Carla Dias	Goiânia	2	255.75
5	7	Maria	Aparecida de Goiânia	1	99.99
6	2	Bruno Costa	Anápolis	1	320.50
7	4	Diego Reis	Aparecida de Goiânia	1	99.99